R GIS Transportation Data Science

Analyzing the Utah Bikeways data

Comparing the bikeways by type with the highway mileage for each city

This analysis calculates and compares roadway lane miles and bikeway facility miles across Utah municipalities using spatial data from the Utah Geospatial Resource Center (UGRC).

Pukar Bhandari Wasatch Front Regional Council
2025-11-16

Table of contents

1 Introduction 2
2 Setup Environment 2
2.1 Install Packages 2
2.2 Load Packages 2
2.3 Environment Variables 3
3 Read Data from UGRC 3
3.1 Utah City Boundaries 4
3.2 Utah County Boundaries 5
3.3 Utah Roadways 6
3.4 Utah Bikeways 7
4 Road Lane Miles by Municipality 8
4.1 Add Unincorporated Areas 8
4.2 Calculate Lane Miles 9
4.3 Aggregate by Municipality 11
5 Bikeway Miles by Municipality 12
5.1 Reclassify Bikeway Types to Categories . . 12
5.2 Calculate Bikeway Lane Miles by Facility Type 14
6 Combine dataframes 17
6.1 Create Multimodal Summary 17
7 Export Final results 21
8 Conclusion 22
Bibliography 22

1 Introduction

This analysis examines the distribution of roadway and bikeway infrastructure across Utah municipalities. The primary objectives are to:

1. Calculate total road lane miles for each municipality
2. Calculate bikeway lane miles by facility type for each municipality
3. Compare roadway and bikeway infrastructure across municipalities

All spatial data is sourced from the Utah Geospatial Resource Center (UGRC) [1] and processed using the R spatial ecosystem. The diagram shows how the pieces fit together.

2 Setup Environment

2.1 Install Packages

The following packages are required for this analysis. Install them if not already available in your R environment.

```
install.packages("dplyr")
install.packages("tidyr")
install.packages("readr")
install.packages("stringr")
install.packages("forcats")
install.packages("fs")
install.packages("sf")
install.packages("arcgislayers")
install.packages("janitor")
install.packages("units")
install.packages("tmap")
```

2.2 Load Packages

```
library(dplyr) # for data manipulation
library(tidyr) # for tidy data operations (pivot
functions)
library(readr) # read and write tabular data
library(stringr) # manipulate strings
library(forcats) # process categorical data
library(fs) # for file system management
```

```
library(sf) # for simple feature geometries
library(arcgislayers) # to read arcgis rest services
library(janitor) # clean column names
library(units) # set and adjust units of measurement
library(tmap) # view sf objects
```

2.3 Environment Variables

We use EPSG:3566 (NAD83(HARN) / Utah North) projection, which uses US survey feet as the base unit. This projection is appropriate for statewide analysis in Utah and ensures accurate distance calculations.

```
PROJECT_CRS = "EPSG:3566"

tmap::tmap_mode("view")
```

Line 1
Every layer is reprojected to this CRS, so lengths come out in US survey feet.

Line 3
`tmap` draws interactive maps in the web version of this post.

3 Read Data from UGRC

All spatial datasets are downloaded from UGRC's ArcGIS REST services. To improve performance and enable offline work, the data is cached locally as GeoDatabase files. Each file is downloaded the first time the post is rendered and reused afterwards, so nothing needs to be fetched by hand (the statewide roads file is large and is not committed).

Listing 1: A retry wrapper for paged ArcGIS downloads

```
dir_data ← "_data"
fs::dir_create(dir_data)

arc_read_retry ← function(url, ..., tries = 5) {
  for (i in seq_len(tries)) {
    out ←
tryCatch(arcgislayers::arc_read(url, ...), error =
function(e) e)
    if (!inherits(out, "error")) return(out)
    if (i == tries) stop(out)
    message("Download failed (attempt ", i, " of ",
tries, "), retrying...")
    Sys.sleep(5 * i)
  }
}
```

Line 1
Downloaded layers are cached in this folder.

Line 2
Created on the first render, if it does not exist yet.

Line 5
UGRC downloads are paged (the roads layer is over 200 pages), so one dropped connection fails the whole read. Try up to `tries` times before giving up.

Line 6
Catch the error instead of stopping, so the loop can decide what to do next.

Line 7
A successful read is returned straight away.

Line 8
After the last attempt, raise the error.

Line 10
Wait a little longer after each failure.

Data used

- Utah Roads
- Bikeways
- Municipal boundaries
- County boundaries

Retried downloads are described in Listing 1.

3.1 Utah City Boundaries

Municipal boundaries are used to aggregate roadway and bikeway data by city. These boundaries represent the legal limits of incorporated municipalities in Utah.

```
path_ut_cities ← file.path(dir_data,
"UtahMunicipalBoundaries.gpkg")
```

```
if (!fs::file_exists(path_ut_cities)) {
  sf_ut_cities ← arc_read_retry(
    "https://services1.arcgis.com/99lidPhWCzftIe9K/
ArcGIS/rest/services/UtahMunicipalBoundaries/
FeatureServer/0",
    crs = PROJECT_CRS
  ) ▷ janitor::clean_names()
  sf_ut_cities ▷
    sf::st_write(path_ut_cities, layer =
"UtahMunicipalBoundaries", append = FALSE)
}

sf_ut_cities ← sf::st_read(path_ut_cities, quiet =
TRUE) ▷
  sf::st_transform(PROJECT_CRS)
```

Line 1
Where this layer is cached.

Line 3
Download only if there is no local copy yet.

Line 4
Read the feature service with the retry wrapper from Listing 1.

Line 9
Write it to a GeoPackage for next time.

Line 12
Always read from the local copy, so the geometry column is the same whether the file was just downloaded or already existed.

The same download-once pattern repeats for the county, roads and bikeways layers below.

```
tmap::qtm(sf_ut_cities)
```

3.2 Utah County Boundaries

County boundaries are used to aggregate roadway and bikeway data for unincorporated areas in Utah.

```
# Define paths
path_ut_counties ← file.path(dir_data,
"UtahCountyBoundaries.gpkg")

# Download if not exists
if (!fs::file_exists(path_ut_counties)) {
  # Read data using arcgis package
  sf_ut_counties ← arc_read_retry(
```

```
    "https://services1.arcgis.com/99lidPhWCzftIe9K/
ArcGIS/rest/services/UtahCountyBoundaries/
FeatureServer/0",
    crs = PROJECT_CRS
  ) ▷ janitor::clean_names()
  # Write to a GeoDatabase
  sf_ut_counties ▷
    sf::st_write(path_ut_counties, layer =
"UtahCountyBoundaries", append = FALSE)
}

# Always read from the local copy, so the geometry
column is the same
# whether the file was just downloaded or already
existed
sf_ut_counties ← sf::st_read(path_ut_counties,
quiet = TRUE) ▷
  sf::st_transform(PROJECT_CRS)
```

```
tmap::qtm(sf_ut_cities)
```

3.3 Utah Roadways

The Utah Roads dataset contains all roadway centerlines statewide, including the number of through lanes for each road segment. This forms the basis for calculating road lane miles.

```
# Define paths
path_ut_roads ← file.path(dir_data,
"UtahRoads.gpkg")

# Download if not exists
if (!fs::file_exists(path_ut_roads)) {
  # Read data using arcgis package
  sf_ut_roads ← arc_read_retry(
    "https://services1.arcgis.com/99lidPhWCzftIe9K/
ArcGIS/rest/services/UtahRoads/FeatureServer/0",
    crs = PROJECT_CRS
  ) ▷ janitor::clean_names()
  # Write to a GeoDatabase
  sf_ut_roads ▷
    sf::st_write(path_ut_roads, layer = "UtahRoads",
append = FALSE)
}

# Always read from the local copy, so the geometry
column is the same
# whether the file was just downloaded or already
existed
```

```
sf_ut_roads ← sf::st_read(path_ut_roads, quiet =
TRUE) ▷
  sf::st_transform(PROJECT_CRS)
```

```
tmap::qtm(sf_ut_roadways)
```

3.4 Utah Bikeways

The Bikeways dataset contains all existing bicycle facilities in Utah. Each segment has two facility type attributes (`facility1` and `facility2`) representing the bicycle facility in each direction of travel.

```
# Define paths
path_ut_bikeways ← file.path(dir_data,
"Bikeways.gpkg")

# Download if not exists
if (!fs::file_exists(path_ut_bikeways)) {
  # Read data using arcgis package
  sf_ut_bikeways ← arc_read_retry(
    "https://services1.arcgis.com/99lidPhWCzftIe9K/
ArcGIS/rest/services/Bikeways/FeatureServer/0",
    crs = PROJECT_CRS
  ) ▷ janitor::clean_names()
  # Write to a GeoDatabase
  sf_ut_bikeways ▷
    sf::st_write(path_ut_bikeways, layer =
"Bikeways", append = FALSE)
}

# Always read from the local copy, so the geometry
column is the same
# whether the file was just downloaded or already
existed
sf_ut_bikeways ← sf::st_read(path_ut_bikeways,
quiet = TRUE) ▷
  sf::st_transform(PROJECT_CRS)
```

```
tmap::qtm(sf_ut_bikeways)
```

4 Road Lane Miles by Municipality

4.1 Add Unincorporated Areas

```
tmap::qtm(sf_ut_cities, col = "blue", fill = NULL,
size = 1) +
  tmap::qtm(sf_ut_counties, col = "red", fill =
NULL, size = 2)
```

```
# Create combined municipal boundaries including
unincorporated areas
sf_ut_municipal ← dplyr::bind_rows(
  sf_ut_cities ▷
    sf::st_make_valid() ▷
    dplyr::select(Name = name),

  sf_ut_counties ▷
    sf::st_make_valid() ▷
    dplyr::mutate(
      Name = paste0(stringr::str_to_title(name), "
County (Unincorporated)")
    ) ▷
    dplyr::select(Name) ▷
    sf::st_difference(
      sf_ut_cities ▷
        sf::st_make_valid() ▷
        sf::st_union() ▷
        sf::st_make_valid()
    )
)

sf_ut_municipal
```

Line 4
Incorporated cities come first; `st_make_valid()` repairs any broken polygons before they are combined.

Line 10
Each county becomes an “unincorporated” area, with its name in proper case.

Line 13
Cut out everything that falls inside a city, so only the unincorporated remainder is left.

```
Simple feature collection with 290 features and 1
field
```

```
Geometry type: GEOMETRY
Dimension:     XY
Bounding box:  xmin: 895000.7 ymin: 6076316 xmax:
2358017 ymax: 7905124
Projected CRS: NAD83 / Utah Central (ftUS)
First 10 features:
          Name                           geom
1       Newton POLYGON ((1503755 7845226, ...
2       Eureka POLYGON ((1472271 7153867, ...
3   Huntsville POLYGON ((1566912 7624813, ...
4   Springdale POLYGON ((1209138 6151420, ...
5  Grantsville POLYGON ((1349646 7393972, ...
6    Bluffdale POLYGON ((1516737 7350351, ...
7     Blanding POLYGON ((2224889 6305132, ...
8     Herriman POLYGON ((1484817 7363912, ...
9        Levan POLYGON ((1539952 7009592, ...
10   Panguitch POLYGON ((1368834 6382054, ...
```

```
tmap::qtm(sf_ut_municipal, fill = NULL)
```

4.2 Calculate Lane Miles

Road lane miles are calculated by multiplying the length of each road segment by the number of through lanes.

> **⚠ Missing lane counts are assumed to be one lane**
>
> Where a road segment's through-lane count is missing (NA), the calculation assumes 1 lane as a conservative estimate. Fixing this in the source data is still a to-do.

The calculation converts the geometry length from US survey feet to miles using the `units` package, which properly handles unit conversions and maintains unit awareness throughout the analysis.

```
sf_ut_roads ← sf_ut_roads ▷
    dplyr::mutate(
        dot_thrulanes =
dplyr::if_else(is.na(dot_thrulanes), 1,
dot_thrulanes),
        miles =
units::set_units(sf::st_length(`geom`), "mile"),
        lane_miles = units::set_units(dot_thrulanes
```

```
* miles, "mile")
    )

sf_ut_roads ▷
  dplyr::select(dot_thrulanes, miles, lane_miles) ▷
  head(20)
```

Line 3
A missing through-lane count is treated as one lane (to do: fix this in the source data).

Line 4
Segment length, converted from US survey feet to miles.

Line 5
Lane miles are the length times the number of through lanes.

```
Simple feature collection with 20 features and 3
fields
Geometry type: MULTILINESTRING
Dimension:     XY
Bounding box:  xmin: 1525086 ymin: 6800123 xmax:
1927447 ymax: 7058250
Projected CRS: NAD83 / Utah Central (ftUS)
First 10 features:
   dot_thrulanes             miles        lane_miles
1              1 1.52909874 [mile] 1.52909874 [mile]
2              1 2.86144968 [mile] 2.86144968 [mile]
3              1 0.31686554 [mile] 0.31686554 [mile]
4              1 1.06293998 [mile] 1.06293998 [mile]
5              1 2.70255959 [mile] 2.70255959 [mile]
6              1 0.80867386 [mile] 0.80867386 [mile]
7              1 0.71336950 [mile] 0.71336950 [mile]
8              1 0.36598670 [mile] 0.36598670 [mile]
9              1 0.09770893 [mile] 0.09770893 [mile]
10             1 0.25556167 [mile] 0.25556167 [mile]
                            geom
1  MULTILINESTRING ((1580305 6 ...
2  MULTILINESTRING ((1582439 6 ...
3  MULTILINESTRING ((1720276 6 ...
4  MULTILINESTRING ((1585991 6 ...
5  MULTILINESTRING ((1918710 6 ...
6  MULTILINESTRING ((1655301 7 ...
7  MULTILINESTRING ((1548359 6 ...
8  MULTILINESTRING ((1614298 6 ...
9  MULTILINESTRING ((1606261 7 ...
10 MULTILINESTRING ((1606826 6 ...
```

4.3 Aggregate by Municipality

Using spatial join, we associate each road segment with its municipality and sum the lane miles. Roads that fall outside municipal boundaries will have NA for city_name.

```
# Calculate total lane miles by municipality
road_miles_by_city ← sf_ut_roads ▷
    sf::st_join(
      sf_ut_municipal,
      join = sf::st_intersects
    ) ▷
    dplyr::group_by(Name) ▷
    dplyr::summarize(
      total_lane_miles = sum(lane_miles, na.rm =
TRUE),
      .groups = "drop"
    ) ▷
    sf::st_drop_geometry()

road_miles_by_city ▷
  dplyr::arrange(dplyr::desc(total_lane_miles))
```

Line 3
Spatial join: attach each road segment to the city or unincorporated area it falls within.

Line 7
Group by municipality and add up the lane miles.

Line 12
Keep a plain table; the geometry is no longer needed.

```
# A tibble: 287 × 2
   Name
total_lane_miles
   <chr>
[mile]
 1 Millard County (Unincorporated)
8827.
 2 San Juan County (Unincorporated)
8138.
 3 Grand County (Unincorporated)
7153.
 4 Uintah County (Unincorporated)
6588.
 5 Box Elder County (Unincorporated)
5613.
 6 Tooele County (Unincorporated)
```

```
5593.
 7 Iron County (Unincorporated)
5529.
 8 Garfield County (Unincorporated)
4871.
 9 Emery County (Unincorporated)
4680.
10 Juab County (Unincorporated)
4583.
# i 277 more rows
```

Key finding

Salt Lake City has the highest road lane miles among Utah municipalities, followed by St. George and West Valley City. The NA category represents roads outside municipal boundaries (unincorporated areas, state highways between cities, etc.).

5 Bikeway Miles by Municipality

5.1 Reclassify Bikeway Types to Categories

We use `forcats` package to respect the hierarchy of the bicycle facilities.

```
facility_class_order ← c(
  "Paved Path",
  "Protected Bike Lane",
  "Bike Lane",
  "Marked Route",
  "Unmarked Route",
  "Other"
)

sf_ut_bikeways ← sf_ut_bikeways ▷
  dplyr::mutate(
    facility1_class = dplyr::case_when(
      facility1 == "Trail or Pathway" ~ "Paved
Path",
      facility1 %in% c(
        "Cycle track, at-grade, protected with
```

```
parking (1A)",
        "Cycle track, protected with barrier (1B)",
        "Cycle track, raised and curb separated
(1C)"
      ) ~ "Protected Bike Lane",
      facility1 %in% c(
        "Buffered bike lane (2A)",
        "Bike lane (2B)"
      ) ~ "Bike Lane",
      facility1 %in% c(
        "Marked shared roadway (3B)",
        "Signed shared roadway (3C)"
      ) ~ "Marked Route",
      facility1 == "Shoulder bikeway (3A)" ~
"Unmarked Route",
      TRUE ~ "Other"
    ),
    facility2_class = dplyr::case_when(
      facility2 == "Trail or Pathway" ~ "Paved
Path",
      facility2 %in% c(
        "Cycle track, at-grade, protected with
parking (1A)",
        "Cycle track, protected with barrier (1B)",
        "Cycle track, raised and curb separated
(1C)"
      ) ~ "Protected Bike Lane",
      facility2 %in% c(
        "Buffered bike lane (2A)",
        "Bike lane (2B)"
      ) ~ "Bike Lane",
      facility2 %in% c(
        "Marked shared roadway (3B)",
        "Signed shared roadway (3C)"
      ) ~ "Marked Route",
      facility2 == "Shoulder bikeway (3A)" ~
"Unmarked Route",
      TRUE ~ "Other"
    ),
    facility1_class = factor(facility1_class, levels
= facility_class_order),
    facility2_class = factor(facility2_class, levels
= facility_class_order)
  )
```

Line 1

The classes, from the most protected facility to the least, in the order `forcats` will respect.

Line 12
Collapse UGRC's detailed facility names into five broader classes; the same recoding is applied to `facility2` below.

Line 48
Turn both columns into ordered factors, so tables and plots keep this order.

5.2 Calculate Bikeway Lane Miles by Facility Type

Bikeway facilities are directional - a single bikeway segment can have different facility types in each direction (`facility1` and `facility2`).

> **Bikeway miles are lane miles, counted per direction**
>
> Each direction is treated as a separate lane, so a 1-mile bikeway segment with bike lanes in both directions counts as 2 lane-miles of bike lanes.

The `pivot_longer()` function stacks `facility1` and `facility2` into separate rows, effectively doubling the mileage for segments where both directions have facilities. We then group by municipality and facility type to calculate total lane miles for each combination.

```
# Prepare bikeways with both directions as lanes
bikeway_miles_by_city ← sf_ut_bikeways ▷
    dplyr::filter(cartocode ≠ "9") ▷
    dplyr::select( facility1_class, facility2_class)
▷

    # Add mileage to each bikeway segment
    dplyr::mutate(
      miles = units::set_units(sf::st_length(geom),
"mile")
    ) ▷

    sf::st_join(
      sf_ut_municipal,
      join = sf::st_intersects
    ) ▷

    # Drop geometry from dataframe for further
```

```
processing
    sf::st_drop_geometry() ▷

    tidyr::pivot_longer(
      cols = c(facility1_class, facility2_class),
      values_to = "facility_class"
    ) ▷

    dplyr::mutate(
      weighted_miles = dplyr::if_else(
        facility_class == "Paved Path",
        as.numeric(miles) * 0.5, # TODO: change this
to 1 if needed
        as.numeric(miles) * 0.5
      )
    ) ▷

    # Summarize total lane miles by city and
facility class
    dplyr::group_by(Name, facility_class) ▷
    dplyr::summarize(
      total_lane_miles =
sum(as.numeric(weighted_miles), na.rm = TRUE),
      .groups = "drop"
    ) ▷

    # Make facility class into columns
    tidyr::pivot_wider(
      names_from = facility_class,
      values_from = total_lane_miles,
      values_fill = 0
    )

# Add total across all bike lane types
bikeway_miles_by_city ← bikeway_miles_by_city ▷
    dplyr::rowwise() ▷
    dplyr::mutate(
      `All Biking Facilities` =
sum(dplyr::c_across(!Name))
    ) ▷
    dplyr::ungroup()

# View the results
bikeway_miles_by_city ▷
  dplyr::arrange(dplyr::desc(`All Biking
Facilities`))
```

Line 3

Drop “sidewalk” and “virtual link” connections.

Line 19
Treat `facility1` and `facility2` as separate lane directions.

Line 24
Each direction counts for half the segment length. Paths could be weighted 1.0 instead, which is the to-do in the code.

```
# A tibble: 287 × 8
   Name           `Paved Path` `Bike Lane` `Unmarked
Route` Other `Marked Route`
   <chr>                 <dbl>       <dbl>
<dbl> <dbl>          <dbl>
 1 Garfield Coun…        17.3         0.954
55.1 2611.          0
 2 San Juan Coun…        0.483       0
130.  2218.          0
 3 Millard Count…        0           0
40.7 2007.          0
 4 Grand County …       11.4         0.257
42.0 1435.          0
 5 Uintah County…        0           1.44
114.  1328.          0
 6 Box Elder Cou…        0           0
33.6 1285.          0
 7 Tooele County…        2.75        1.72
52.6 1227.          0
 8 Duchesne Coun…        0           0
54.1 1168.          0.700
 9 Juab County (…        0           0
45.1 1125.          0
10 Beaver County…        2.05        0
22.7 1069.          0
# ℹ 277 more rows
# ℹ 2 more variables: `Protected Bike Lane` <dbl>,
#   `All Biking Facilities` <dbl>
```

Key finding

Bikeway infrastructure varies significantly by facility type across municipalities. The dataset includes various facility types such as bike lanes, protected bike lanes, shared-use paths, and trails.

6 Combine dataframes

6.1 Create Multimodal Summary

The final step combines road lane miles and bikeway lane miles by facility type into a single dataset. This allows for direct comparison of roadway and bikeway infrastructure across municipalities.

Each row represents a municipality, with columns for total road lane miles followed by columns for each bikeway facility type. Cities with zero bikeway miles for a particular facility type will show 0 in that column.

```
# Combine with roads
combined_miles_by_city ← road_miles_by_city ▷
  # Convert road lane miles to numeric
  dplyr::mutate(`All Roads` =
as.numeric(total_lane_miles)) ▷
  dplyr::select(Name, `All Roads`) ▷
  # Join with bikeway data
  dplyr::right_join(bikeway_miles_by_city, by =
"Name") ▷
  # Reorder columns: Name, bike categories, total
bike, then road
  dplyr::select(
    Name,
    dplyr::all_of(facility_class_order),   # bike
lane categories
    - Other,
    # `All Biking Facilities`,
    `All Roads`
  ) ▷
  # Round all numeric columns to one decimal
  dplyr::mutate(dplyr::across(where(is.numeric),
\(x) round(x, 1)))

combined_miles_by_city
```

```
# A tibble: 287 × 7
   Name          `Paved Path` `Protected Bike Lane`
`Bike Lane` `Marked Route`
   <chr>                <dbl>                 <dbl>
<dbl>          <dbl>
 1 Alpine                15.1                     0
0.7              0
 2 Alta                  0                       0
```

```
0               1
 3 Altamont               0                       0
0               0
 4 Alton                  0                       0
0               0
 5 Amalga                 0.4                     0
0               0
 6 American Fork          9.7                     2
6.9             0
 7 Annabella              0                       0
0               0
 8 Antimony               0                       0
0               0
 9 Apple Valley           0                       0
0               0
10 Aurora                 0                       0
0               0
# ℹ 277 more rows
# ℹ 2 more variables: `Unmarked Route` <dbl>, `All
Roads` <dbl>
```

```
//| echo: false

// 1. Multi-select Dropdown for Bikeway Types
viewof bikeTypes = Inputs.select(
  ["Paved Path", "Protected Bike Lane", "Bike Lane",
"Marked Route", "Unmarked Route"],
  {
    value: ["Paved Path", "Protected Bike Lane"], //
High-quality defaults
    label: "Select Facility Types:",
    multiple: true // Enables the multi-select
dropdown behavior
  }
)

// 2. City Search with robust error handling
viewof searchCity = Inputs.text({placeholder:
"Search City (e.g. Salt Lake City)", label:
"Highlight City"})

// 3. Dynamic Data Processing
processedData = {
  const selectedTypes = bikeTypes;
  const data = transpose(miles_data)
    // Filter out rows with missing names or zero
road miles
    .filter(d => d.Name && d.Name !== "NA" && d["All
Roads"] > 0)
    .map(d => {
      const roadMiles = d["All Roads"] || 0;
```

```
      const selectedBikeMiles =
selectedTypes.reduce((acc, type) => acc +
(parseFloat(d[type]) || 0), 0);
      const ratio = (selectedBikeMiles / roadMiles)
* 100;

      return {
        ...d,
        selectedBikeMiles,
        ratio
      };
    });

  const avgRatio = d3.mean(data, d => d.ratio);

  return data.map(d => ({
    ...d,
    avgRatio,
    maturity: d.ratio >= avgRatio ? "Above
Average" : "Below Average"
  }));
}

// 4. Highlight Logic
highlighted = processedData.filter(d => {
  if (!searchCity || !d.Name) return false;
  return
d.Name.toLowerCase().includes(searchCity.toLowerCase());
})

// 5. The Visualization
Plot.plot({
  title: "Utah Bikeway Maturity Index",
  subtitle: "Comparison of selected bikeway miles
vs. total road network size.",
  caption: "The dashed line represents the statewide
average ratio for the selected infrastructure
types.",
  width: width,
  height: 600,
  grid: true,
  style: {
    fontSize: "12px",
    fontFamily: "sans-serif"
  },
  x: { label: "Total Road Miles (Log Scale) →",
type: "log", domain: [10, 5000] },
  y: { label: "Selected Bikeway Miles (Log Scale)
→", type: "log", domain: [0.1, 500] },
  style: { backgroundColor: 'transparent' },
  color: {
```

```
    domain: ["Above Average", "Below Average"],
    range: ["#008080", "#CD5C5C"],
    legend: true
  },
  marks: [
    // Average Maturity Benchmark Line
    Plot.line(d3.range(10, 5000, 100), {
      x: d => d,
      y: d => d * (processedData[0].avgRatio / 100),
      stroke: "#444",
      strokeDasharray: "4,4",
      strokeWidth: 1,
      opacity: 0.4
    }),

    // Data Points
    Plot.dot(processedData, {
      x: "All Roads",
      y: "selectedBikeMiles",
      stroke: "maturity",
      fill: "maturity",
      fillOpacity: 0.3,
      r: 4,
      tip: true,
      title: d => `${d.Name}\nBikeway:
${d.selectedBikeMiles.toFixed(1)} mi\nRoadway:
${d["All Roads"]} mi\nRatio: ${d.ratio.toFixed(1)}%`
    }),

    // Highlight the Selected City
    Plot.dot(highlighted, {
      x: "All Roads",
      y: "selectedBikeMiles",
      stroke: "black",
      strokeWidth: 2,
      r: 8
    }),

    // FIXED LABEL: Explicit dark fill with white
halo
    Plot.text(highlighted, {
      x: "All Roads",
      y: "selectedBikeMiles",
      text: "Name",
      dy: -20,
      fontWeight: "bold",
      fontSize: 14,
      fill: "#333", // Explicit dark charcoal
      stroke: "white", // White halo
      strokeWidth: 4,  // Thick enough to be seen
      paintOrder: "stroke"
```

```
    }),

    // Auto-label high performers
    Plot.text(processedData, {
      filter: d ⇒ (d.ratio >
processedData[0].avgRatio * 3 && d["All Roads"] >
100) || d.selectedBikeMiles > 120,
      x: "All Roads",
      y: "selectedBikeMiles",
      text: "Name",
      dy: -10,
      fontSize: 10,
      fill: "#555",
      opacity: 0.8
    })
  ]
})
```

Key finding

The ratio of bikeway lane miles to road lane miles varies considerably across Utah municipalities, indicating different levels of investment in bicycle infrastructure relative to overall roadway networks.

7 Export Final results

The analysis results are exported as a CSV file for further use in reports, visualizations, or other analyses. The output directory is created automatically if it doesn't exist.

```
# set data folder
dir_output ← "_output"
fs::dir_create(dir_output) # Create a directory if
it doesnt exist
```

```
combined_miles_by_city ▷
    readr::write_csv(
        file.path(dir_output, "miles_by_city.csv"),
        append = FALSE
    )
```

Download the output files:

miles_by_city.csv

8 Conclusion

This analysis provides a comprehensive inventory of roadway and bikeway infrastructure across Utah municipalities. The methodology is reproducible and can be updated as new data becomes available from UGRC. The spatial join approach ensures accurate attribution of infrastructure to municipalities, though users should note that roads and bikeways in unincorporated areas will appear under the NA category.

Bibliography

[1] Utah Geospatial Resource Center, "Utah SGID open data: Roads, Bikeways, Municipal Boundaries and County Boundaries." Accessed: Oct. 23, 2025. [Online]. Available: https://gis.utah.gov/